



OpenEden Prism Audit Report

Apr 1, 2026





Table of Contents

Summary	2
Overview	3
Issues	4
[WP-M1] Banned or De-KYC'd Users' Pending Redemptions Become Permanently Stuck in Express	4
[WP-M2] The redemptionQueue ID may collide.	7
[WP-M3] The Express contract lacks a way to modify the _mintMinimum storage slot after initialization.	10
[WP-M4] The protocol requires lastActionBlock != block.number for stake/unstake operations. An attacker can use stakeFor to transfer 1 wei to a user within the same block, updating the user's lastActionBlock and causing subsequent stake/unstake attempts by the user to revert.	12
[WP-M5] The Express.previewRedeem() return value is incorrect. Given that previewRedeem(uint256 _amount) returns (uint256 feeAmt, uint256 redeemAmt) expects the input in units of token wei, the returned values should be in units of underlying wei.	14
[WP-M6] The lastActionBlock[_user] != block.number restriction can be bypassed by transferring vault tokens to a different account.	19
[WP-L7] Missing RedemptionQueue.getPendingRedemptions(user, start, count) .	23
[WP-I8] Out-of-bounds indices should typically trigger an error rather than returning 0.	25
[WP-I9] Consider adding the onlyInitializing modifier to MintRedeemLimiter.__MintRedeemLimiter_init() .	27
[WP-G10] For storage variables that are initialized once and never modified, consider using immutable variables to avoid unnecessary SLOAD gas costs.	29
[WP-N11] The codebase contains logic to write to Express.redemptionContract , but lacks any corresponding read operations.	32
[WP-N12] Unused errors	33



[WP-I13] offRamp Can Withdraw Liquidity Committed to Pending Redemptions	35
Appendix	37
Disclaimer	38



Summary

This report has been prepared for OpenEden Prism smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



Overview

Project Summary

Project Name	OpenEden Prism
Codebase	https://github.com/OpenEdenHQ/openeden.prism.audit/
Commit	0e0387d22eaea73b2835fac539f84ef8811d2864
Language	Solidity

Audit Summary

Delivery Date	Apr 1, 2026
Audit Methodology	Static Analysis, Manual Review
Total Issues	13



[WP-M1] Banned or De-KYC'd Users' Pending Redemptions Become Permanently Stuck in `Express`

Medium

Issue Description

When a user calls `redeemRequest()`, their PRISM tokens are transferred into the `Express` contract and added to the redemption queue. If the user is subsequently banned (in `Token.sol`) or de-KYC'd (in `Express.kycList`) before their request is processed, the funds become permanently unrecoverable:

- `processRedemptionQueue` checks `kycList[sender] && kycList[receiver]` at processing time and reverts if either is false.
- `cancel` returns PRISM to `sender`, but `Token._update` reverts with `BannedRecipient` if `sender` is banned, causing the entire batch cancellation to revert.
- **No emergency recovery function** exists in `Express` to extract stuck PRISM or skip a problematic queue entry.

Because the redemption queue is strictly FIFO (via `DoubleQueueModified`), a single stuck entry at the front blocks all subsequent processing and cancellations for the entries ahead of it.

PoC

1. Alice (KYC'd, not banned) calls `redeemRequest(Alice, 10_000e18)`. PRISM is transferred to `Express`; Alice's entry is added to the front of the queue.
2. Compliance team bans Alice in `Token.sol` (e.g., OFAC sanction).
3. Operator calls `processRedemptionQueue(1)`:
 - KYC check passes (Alice may still be KYC'd).
 - `_distributeToken(underlying, Alice, ...)` — underlying token transfer succeeds (no ban check on underlying).
 - `token.burn(address(this), amount)` — succeeds.
 - **OR** if Alice is also de-KYC'd: `kycList[Alice] == false` → revert `NotInKycList`.
4. Maintainer calls `cancel(1)`:
 - `IERC20(token).safeTransfer(Alice, amount)` → `Token._update` → `BannedRecipient(Alice)` → **revert**.
5. All subsequent queue entries are now blocked.



Vulnerable Code

contracts/extension/Express.sol#L457-L508 — `_processRedemptionQueueInternal` :

```
if (!kycList[sender] || !kycList[receiver])  
    revert NotInKycList(sender, receiver);
```

contracts/extension/Express.sol#L515-L537 — `cancel` :

```
IERC20(address(token)).safeTransfer(sender, amount); // reverts if sender is banned
```

contracts/core/Token.sol#L256-L262 — `_update` :

```
if (to != address(0) && isBanned(to)) {  
    revert BannedRecipient(to);  
}
```

Recommended Fix

1. Add an emergency sweep function restricted to `DEFAULT_ADMIN_ROLE` that can remove a specific queue entry and send PRISM to an admin-controlled address (e.g., a compliance escrow):

```
function emergencyDequeue(uint256 index, address escrow)  
    external  
    onlyRole(DEFAULT_ADMIN_ROLE)  
{  
    // pop the entry at `index`, transfer amount to escrow  
}
```

1. Alternatively, in `cancel`, allow skipping entries where the recipient is banned, returning funds to an admin escrow rather than the banned address.
2. At minimum, document that banned/de-KYC'd users must have their entries manually resolved via a contract upgrade.



Status

✓ Fixed



[WP-M2] The `redemptionQueue` ID may collide.

Medium

Issue Description

PoC:

Assume the current `redemptionQueue.length()` is 5.

Two `redeemRequest` transactions with identical parameters are submitted.

Within the same second, the `OPERATOR` happens to call `processRedemptionQueue`.

Execution order:

1. `redeemRequest` : `length` changes from 5 to 6, `id = hash(***5)`.
2. `processRedemptionQueue` : `length` changes from 6 to 5.
3. `redeemRequest` : `length` changes from 5 to 6, `id = hash(***5)`.

This results in both transactions sharing the same ID.

This is more likely to occur on blockchains with sub-second block times.

This edge case can cause the off-chain `OPERATOR` to record incorrect accounting data.

```
384     function redeemRequest(  
385         address _to,  
386         uint256 _amount  
387     ) external whenNotPausedRedeem {  
388         address from = _msgSender();  
389         if (!kycList[from] || !kycList[_to]) revert NotInKycList(from, _to);  
390         if (_amount == 0) revert InvalidAmount();  
391         if (_amount < _redeemMinimum)  
392             revert RedeemLessThanMinimum(_amount, _redeemMinimum);  
393  
394         IERC20(address(token)).safeTransferFrom(from, address(this), _amount);  
395         redemptionInfo[_to] += _amount;  
396  
397         bytes32 id = keccak256(  
398             abi.encode(  

```

```

399         from,
400         _to,
401         _amount,
402         block.timestamp,
403         redemptionQueue.length()
404     )
405 );
406 bytes memory data = abi.encode(from, _to, _amount, id);
407 redemptionQueue.pushBack(data);
408
409 emit AddToRedemptionQueue(from, _to, _amount, id);
410 }

```

```

457 function _processRedemptionQueueInternal(
458     uint256 _len,
459     uint256 _price
460 ) private {
461     uint256 length = redemptionQueue.length();
462     if (length == 0) revert EmptyQueue();
463     if (_len > length) revert InvalidInput(_len);
464     if (_len == 0) _len = length;
465
466     for (uint256 count = 0; count < _len; ) {
467         bytes memory data = redemptionQueue.front();
468         (
469             address sender,
470             address receiver,
471             uint256 amount,
472             bytes32 prevId
473         ) = _decodeData(data);
474
475         if (!kycList[sender] || !kycList[receiver])
476             revert NotInKycList(sender, receiver);
477
478         uint256 underlyingAmt = convertToUnderlying(underlying, amount)
479             .mulDiv(_price, PRICE_BASE);
480
481         uint256 availableLiquidity = getTokenBalance(address(underlying));
482
483         if (underlyingAmt > availableLiquidity) {
484             break;

```

```
485     }
486
487     uint256 feeAmt = txsFee(underlyingAmt, TxType.REDEEM);
488     uint256 receiveAmt = underlyingAmt - feeAmt;
489
490     redemptionQueue.popFront();
491
492     unchecked {
493         ++count;
494         redemptionInfo[receiver] -= amount;
495     }
496
497     token.burn(address(this), amount);
498
499     _distributeToken(underlying, receiver, receiveAmt, feeAmt);
500     emit ProcessRedeem(
501         sender,
502         receiver,
503         amount,
504         receiveAmt,
505         feeAmt,
506         prevId
507     );
508 }
509 }
```

Status

✓ Fixed



[WP-M3] The `Express` contract lacks a way to modify the `_mintMinimum` storage slot after initialization.

Medium

Issue Description

There is no privileged `setRedeemMinimum` function. Consequently, the `redeemMinimum` value remains permanently fixed at its initial state and cannot be adjusted.

```
58     function _setRedeemMinimum(uint256 redeemMinimum) internal {
59         _redeemMinimum = redeemMinimum;
60         emit RedeemMinimumUpdated(redeemMinimum);
61     }
```

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L655-L695>

```
655     /*//////////////////////////////////////////////////////////////
656                                     MINT/REDEEM LIMITER FUNCTIONS
657     //////////////////////////////////////////////////////////////////////////*/
658
659     /**
660      * @notice Set mint minimum amount (in token, 18 decimals)
661      * @param _mintMinimum Minimum mint amount
662      */
663     function setMintMinimum(
664         uint256 _mintMinimum
665     ) external onlyRole(MAINTAINER_ROLE) {
666         _setMintMinimum(_mintMinimum);
667     }
668
669     /**
670      * @notice Set first deposit requirement amount (in token, 18 decimals)
671      * @param _amount Required first deposit amount
672      */
673     function setFirstDepositAmount(
674         uint256 _amount
675     ) external onlyRole(MAINTAINER_ROLE) {
```

```

676     _setFirstDepositAmount(_amount);
677 }
678
679 /**
680  * @notice Update first deposit status for an account
681  * @param _account Account address
682  * @param _flag First deposit status
683  */
684 function updateFirstDeposit(
685     address _account,
686     bool _flag
687 ) external onlyRole(MAINTAINER_ROLE) {
688     if (_account == address(0)) revert InvalidAddress();
689     firstDeposit[_account] = _flag;
690     emit UpdateFirstDeposit(_account, _flag);
691 }
692
693 /*//////////////////////////////////////
694                                TREASURY MANAGEMENT
695 //////////////////////////////////*/

```

Status

✓ Fixed



[WP-M4] The protocol requires `lastActionBlock != block.number` for stake/unstake operations. An attacker can use `stakeFor` to transfer 1 wei to a user within the same block, updating the user's `lastActionBlock` and causing subsequent stake/unstake attempts by the user to revert.

Medium

Issue Description

The `lastActionBlock != block.number` check is no longer necessary given the current withdrawal queue implementation.

Vault.sol

```
118     function stakeFor(  
119         address _user,  
120         uint256 _amount,  
121         uint256 _minSharesOut  
122     ) external returns (uint256) {  
123         if (_amount == 0) revert InvalidAmount();  
124         uint256 shares = _stake(_user, _amount);  
125         if (shares < _minSharesOut)  
126             revert InsufficientOutput(shares, _minSharesOut);  
127         return shares;  
128     }
```

Vault.sol

```
206     function _stake(address _user, uint256 _amount) internal returns (uint256) {  
207         address sender = msg.sender;  
208  
209         // Check flash loan protection for both sender and recipient  
210         if (lastActionBlock[sender] == block.number) revert FlashLoanDetected();  
211         if (lastActionBlock[_user] == block.number) revert FlashLoanDetected();  
212  
213         // Update last action block for both sender and recipient
```



```
214         lastActionBlock[sender] = block.number;
215         lastActionBlock[_user] = block.number;
216
217         uint256 shares = super.deposit(_amount, _user);
218         emit Staked(_user, _amount, shares);
219         return shares;
220     }
```

Status

✓ Fixed



[WP-M5] The `Express.previewRedeem()` return value is incorrect. Given that `previewRedeem(uint256 _amount)` returns `(uint256 feeAmt, uint256 redeemAmt)` expects the input in units of `token wei`, the returned values should be in units of `underlying wei`.

Medium

Issue Description

Compared to `_processRedemptionQueueInternal()`, `previewRedeem()` fails to account for `_price` and `convertToUnderlying()`.

For comparison, `previewMint()` correctly accounts for `convertFromUnderlying()`. This inconsistency between `previewMint()` and `previewRedeem()` is likely to cause misunderstandings or misuse.

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L412-L423>

```
412     /**
413      * @notice Preview redemption amounts
414      * @param _amount The amount of token to redeem
415      * @return feeAmt Platform fee amount in token
416      * @return redeemAmt token amount user receives
417      */
418     function previewRedeem(
419         uint256 _amount
420     ) public view returns (uint256 feeAmt, uint256 redeemAmt) {
421         feeAmt = txsFee(_amount, TxType.REDEEM);
422         redeemAmt = _amount - feeAmt;
423     }
```

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L379-L410>

```

379  /**
380   * @notice Queue a redemption request for manual processing
381   * @param _to The address to receive redeemed token
382   * @param _amount The amount of to redeem
383   */
384   function redeemRequest(
385       address _to,
386       uint256 _amount
387   ) external whenNotPausedRedeem {
388       address from = _msgSender();
389       if (!kycList[from] || !kycList[_to]) revert NotInKycList(from, _to);
390       if (_amount == 0) revert InvalidAmount();
391       if (_amount < _redeemMinimum)
392           revert RedeemLessThanMinimum(_amount, _redeemMinimum);
393
394       IERC20(address(token)).safeTransferFrom(from, address(this), _amount);
395       redemptionInfo[_to] += _amount;
396
397       bytes32 id = keccak256(
398           abi.encode(
399               from,
400               _to,
401               _amount,
402               block.timestamp,
403               redemptionQueue.length()
404           )
405       );
406       bytes memory data = abi.encode(from, _to, _amount, id);
407       redemptionQueue.pushBack(data);
408
409       emit AddToRedemptionQueue(from, _to, _amount, id);
410   }

```

```

429  /**
430   * @notice Process redemption queue at 1:1 price
431   * @param _len Number of redemptions to process (0 = process all)
432   */
433   function processRedemptionQueue(
434       uint256 _len
435   ) external onlyRole(OPERATOR_ROLE) {
436       _processRedemptionQueueInternal(_len, PRICE_BASE);

```

```

437     }
438
439     /**
440      * @notice Process redemption queue with a custom price
441      * @param _len Number of redemptions to process (0 = process all)
442      * @param _price Price multiplier in 8 decimals (1e8 = 1.0)
443      */
444     function processRedemptionQueueWithPrice(
445         uint256 _len,
446         uint256 _price
447     ) external onlyRole(MAINTAINER_ROLE) {
448         if (_price >= PRICE_BASE || _price == 0) revert InvalidAmount();
449         _processRedemptionQueueInternal(_len, _price);
450     }
451
452     /**
453      * @notice Internal function to process redemption queue with price scaling
454      * @param _len Number of redemptions to process (0 = process all)
455      * @param _price Price multiplier in 8 decimals (1e8 = 1.0)
456      */
457     function _processRedemptionQueueInternal(
458         uint256 _len,
459         uint256 _price
460     ) private {
461         uint256 length = redemptionQueue.length();
462         if (length == 0) revert EmptyQueue();
463         if (_len > length) revert InvalidInput(_len);
464         if (_len == 0) _len = length;
465
466         for (uint256 count = 0; count < _len; ) {
467             bytes memory data = redemptionQueue.front();
468             (
469                 address sender,
470                 address receiver,
471                 uint256 amount,
472                 bytes32 prevId
473             ) = _decodeData(data);
474
475             if (!kycList[sender] || !kycList[receiver])
476                 revert NotInKycList(sender, receiver);
477
478             uint256 underlyingAmt = convertToUnderlying(underlying, amount)
479                 .mulDiv(_price, PRICE_BASE);

```

```

480
481         uint256 availableLiquidity = getTokenBalance(address(underlying));
482
483         if (underlyingAmt > availableLiquidity) {
484             break;
485         }
486
487         uint256 feeAmt = txsFee(underlyingAmt, TxType.REDEEM);
488         uint256 receiveAmt = underlyingAmt - feeAmt;
489
490         redemptionQueue.popFront();
491
492         unchecked {
493             ++count;
494             redemptionInfo[receiver] -= amount;
495         }
496
497         token.burn(address(this), amount);
498
499         _distributeToken(underlying, receiver, receiveAmt, feeAmt);
500         emit ProcessRedeem(
501             sender,
502             receiver,
503             amount,
504             receiveAmt,
505             feeAmt,
506             prevId
507         );
508     }
509 }

```

```

@@ 750,756 @@
757     function _distributeToken(
758         address _token,
759         address _to,
760         uint256 _amount,
761         uint256 _feeAmount
762     ) private {
763         if (_feeAmount > 0) {
764             IERC20(_token).safeTransfer(feeTo, _feeAmount);
765         }

```



```
766         IERC20(_token).safeTransfer(_to, _amount);
767     }
```

```
358     /**
359      * @notice Preview mint amounts
360      * @param _underlying The underlying token address
361      * @param _amount The amount of underlying token
362      * @return netAmt Net amount after fees
363      * @return feeAmt Fee amount
364      * @return mintAmt token amount to be minted
365      */
366     function previewMint(
367         address _underlying,
368         uint256 _amount
369     ) public view returns (uint256 netAmt, uint256 feeAmt, uint256 mintAmt) {
370         feeAmt = txsFee(_amount, TxType.MINT);
371         netAmt = _amount - feeAmt;
372         mintAmt = convertFromUnderlying(_underlying, netAmt);
373     }
```

Status

✓ Fixed



[WP-M6] The `lastActionBlock[_user] != block.number` restriction can be bypassed by transferring vault tokens to a different account.

Medium

Issue Description

For example, by transferring vault tokens to a new EOA address, the restriction is avoided because the new address has no `lastActionBlock[_user]` record.

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/core/Vault.sol#L26-L254>

```
26  contract Vault is
27      Initializable,
28      ERC20Upgradeable,
29      ERC20PausableUpgradeable,
30      ERC4626Upgradeable,
31      AccessControlEnumerableUpgradeable,
32      UUPSUpgradeable
33  {
34      @@ 34,45 @@
35
36
37      // Flash loan protection: track last action block per user
38      mapping(address => uint256) private lastActionBlock;
39
40      @@ 50,105 @@
41
42
43      106
44      107      function stake(
45          108          uint256 _amount,
46          109          uint256 _minSharesOut
47      110      ) external returns (uint256) {
48          111          if (_amount == 0) revert InvalidAmount();
49          112          uint256 shares = _stake(msg.sender, _amount);
50          113          if (shares < _minSharesOut)
51          114              revert InsufficientOutput(shares, _minSharesOut);
52          115          return shares;
53          116      }
```

```

117
118     function stakeFor(
119         address _user,
120         uint256 _amount,
121         uint256 _minSharesOut
122     ) external returns (uint256) {
123         if (_amount == 0) revert InvalidAmount();
124         uint256 shares = _stake(_user, _amount);
125         if (shares < _minSharesOut)
126             revert InsufficientOutput(shares, _minSharesOut);
127         return shares;
128     }
129
130     function unstake(uint256 _shares) external returns (uint256) {
131         if (_shares == 0) revert InvalidAmount();
132         return _unstake(msg.sender, _shares);
133     }
134
135 @@ 135,188 @@
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190     function _update(
191         address from,
192         address to,
193         uint256 value
194     ) internal override(ERC20Upgradeable, ERC20PausableUpgradeable) {
195         if (from != address(0)) {
196             if (IToken(asset()).isBanned(from)) revert BannedAddress(from);
197         }
198         if (to != address(0)) {
199             if (IToken(asset()).isBanned(to)) revert BannedAddress(to);
200         }
201
202         if (paused()) revert VaultPausedTransfers();
203         super._update(from, to, value);
204     }
205
206     function _stake(address _user, uint256 _amount) internal returns (uint256) {
207         address sender = msg.sender;
208
209         // Check flash loan protection for both sender and recipient
210         if (lastActionBlock[sender] == block.number) revert FlashLoanDetected();
211         if (lastActionBlock[_user] == block.number) revert FlashLoanDetected();

```

```

212
213     // Update last action block for both sender and recipient
214     lastActionBlock[sender] = block.number;
215     lastActionBlock[_user] = block.number;
216
217     uint256 shares = super.deposit(_amount, _user);
218     emit Staked(_user, _amount, shares);
219     return shares;
220 }
221
222 function _unstake(
223     address _user,
224     uint256 _shares
225 ) internal returns (uint256) {
226     if (lastActionBlock[_user] == block.number) revert FlashLoanDetected();
227     lastActionBlock[_user] = block.number;
228
229     uint256 assets = super.redeem(_shares, redemptionQueue, _user);
230
231     uint256 redemptionId = IRedemptionQueue(redemptionQueue).enqueue(
232         _user,
233         assets,
234         _shares
235     );
236     emit UnstakeRequested(
237         _user,
238         redemptionId,
239         assets,
240         _shares,
241         block.timestamp
242     );
243
244     return assets;
245 }
246
247 @@ 247,253 @@
254 }

```



Status

✓ Fixed



[WP-L7] Missing `RedemptionQueue.getPendingRedemptions(user, start, count)` .

Low

Issue Description

RedemptionQueue L152 states: “Use `getPendingRedemptions(user, start, count)` for large lists”. I expected to find read access to `getPendingRedemptions(user, start, count)` to handle large lists where `getAllPendingRedemptions()` would cause an out-of-gas error.

However, we could not find the implementation for `getPendingRedemptions(user, start, count)` .

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/core/RedemptionQueue.sol#L10-L259>

```
10  /**
11   * @title RedemptionQueue
12   * @notice Manages T+N unstaking queue for Vault
13   * @dev Users queue redemptions, wait N days, then claim token
14   */
15  contract RedemptionQueue is
16      Initializable,
17      AccessControlEnumerableUpgradeable,
18      UUPSUpgradeable
19  {
20      @@ 20,146 @@
147
148      /**
149       * @notice Get all pending redemptions for user (convenience function)
150       * @param user User address
151       * @return Array of all pending redemptions
152       * @dev Use getPendingRedemptions(user, start, count) for large lists
153       */
154      function getAllPendingRedemptions(
155          address user
156      ) external view returns (Redemption[] memory) {
157          uint256 count = redemptionCount[user];
158          uint256 pendingCount;
```

```

159
160     // Count pending redemptions
161     for (uint256 i; i < count; ) {
162         if (!redemptions[user][i].processed) {
163             unchecked {
164                 ++pendingCount;
165             }
166         }
167         unchecked {
168             ++i;
169         }
170     }
171
172     // Populate array
173     Redemption[] memory pending = new Redemption[](pendingCount);
174     uint256 index;
175     for (uint256 i; i < count; ) {
176         if (!redemptions[user][i].processed) {
177             pending[index] = redemptions[user][i];
178             unchecked {
179                 ++index;
180             }
181         }
182         unchecked {
183             ++i;
184         }
185     }
186
187     return pending;
188 }
189
@@ 190,258 @@
259 }

```

Status

 Acknowledged



[WP-I8] Out-of-bounds indices should typically trigger an error rather than returning 0.

Informational

Issue Description

An out-of-bounds access should generally throw an error and revert the transaction, similar to the panic triggered by a DoubleEndedQueue.

<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v5.6.1/contracts/utils/structs/DoubleEndedQueue.sol#L193>

```
185     /**
186      * @dev Return the item at a position in the queue given by `index`, with the
187      * first item at 0 and last item at
188      *
189      * `length(deque) - 1`.
190      *
191      * Reverts with {Panic-ARRAY_OUT_OF_BOUNDS} if the index is out of bounds.
192      */
193     function at(Bytes32Deque storage deque, uint256 index) internal view returns
194     (bytes32) {
195         (bool success, bytes32 value) = tryAt(deque, index);
196         if (!success) Panic.panic(Panic.ARRAY_OUT_OF_BOUNDS);
197         return value;
198     }
```

However, the Express implementation does not revert; it returns 0 instead.

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L539-L560>

```
539     /**
540      * @notice Get redemption queue information at index
541      * @param _index Queue index to query
542      * @return (uint256,uint256,uint256,uint256)
543      */
544     function getRedemptionQueueInfo(uint256 _index) public view returns (uint256,uint256,uint256,uint256) {
545         (uint256 _total, uint256 _available, uint256 _redeemed, uint256 _pending) = _getRedemptionQueueInfo(_index);
546         return (_total, _available, _redeemed, _pending);
547     }
```

```
546     */
547     function getRedemptionQueueInfo(
548         uint256 _index
549     )
550         external
551         view
552         returns (address sender, address receiver, uint256 amount, bytes32 id)
553     {
554         if (redemptionQueue.empty() || _index > redemptionQueue.length() - 1) {
555             return (address(0), address(0), 0, 0x0);
556         }
557
558         bytes memory data = bytes(redemptionQueue.at(_index));
559         (sender, receiver, amount, id) = _decodeData(data);
560     }
```

Status

 Acknowledged



[WP-I9] Consider adding the `onlyInitializing` modifier to `MintRedeemLimiter.__MintRedeemLimiter_init()` .

Informational

Issue Description

Typically, the `_init` function of a parent upgradable contract includes the `onlyInitializing` modifier.

For example:

<https://github.com/OpenZeppelin/openzeppelin-contracts-upgradeable/blob/v5.6.1/contracts/token/ERC20/ERC20Upgradeable.sol#L57-L65>

```
57     function __ERC20_init(string memory name_, string memory symbol_) internal
    onlyInitializing {
58         __ERC20_init_unchained(name_, symbol_);
59     }
60
61     function __ERC20_init_unchained(string memory name_, string memory symbol_)
    internal onlyInitializing {
62         ERC20Storage storage $ = _getERC20Storage();
63         $_name = name_;
64         $_symbol = symbol_;
65     }
```

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/MintRedeemLimiter.sol#L10-L74>

```
10  /**
11   * @title MintRedeemLimiter
12   * @notice contract implementing minimum thresholds for minting and redeeming.
13   */
14  abstract contract MintRedeemLimiter {
15      @@ 15,22 @@
```

```

23
24     /**
25      * @notice Initializes mint and redeem minimums.
26      * @param mintMinimum    Min amount allowed to mint in one transaction (in
token decimals - 18)
27      * @param redeemMinimum  Min amount allowed to redeem in one transaction (in
token decimals - 18)
28      * @param firstDepositAmount The first deposit amount (in token decimals - 18)
29      */
30
31     function __MintRedeemLimiter_init(
32         uint256 mintMinimum,
33         uint256 redeemMinimum,
34         uint256 firstDepositAmount
35     ) internal {
36         _mintMinimum = mintMinimum;
37         _redeemMinimum = redeemMinimum;
38         _firstDepositAmount = firstDepositAmount;
39     }
40
41     @@ 41,73 @@
74 }

```

Status

✓ Fixed



[WP-G10] For storage variables that are initialized once and never modified, consider using `immutable` variables to avoid unnecessary `SLOAD` gas costs.

Gas

Issue Description

For instance, `RedemptionQueue.asset` is not modified outside of `initialize()` ; this should be refactored as an `immutable` variable in the implementation contract.

`Express.token` and `Express.underlying` follow the same pattern as `RedemptionQueue.asset` .

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/core/RedemptionQueue.sol#L10-L259>

```
10  /**
11   * @title RedemptionQueue
12   * @notice Manages T+N unstaking queue for Vault
13   * @dev Users queue redemptions, wait N days, then claim token
14   */
15  contract RedemptionQueue is
16      Initializable,
17      AccessControlEnumerableUpgradeable,
18      UUPSUpgradeable
19  {
20      @@ 20,31 @@
32
33      IERC20 public asset;
34      @@ 34,71 @@
72
73      /**
74       * @notice Initialize the contract
75       * @param _admin Default admin address
76       * @param _asset token address
77       * @param _vault Vault address
78       * @param _delay Processing delay in seconds (e.g., 7 days = 604800)
79       */
80      function initialize(
```

```

81     address _admin,
82     address _asset,
83     address _vault,
84     uint256 _delay
85 ) public initializer {
86     __AccessControlEnumerable_init();
87     __UUPSUpgradeable_init();
88
89     if (_admin == address(0)) revert InvalidAddress();
90     if (_asset == address(0)) revert InvalidAddress();
91
92     _grantRole(DEFAULT_ADMIN_ROLE, _admin);
93     asset = IERC20(_asset);
94     vault = _vault;
95     delay = _delay;
96 }
97
@@ 98,128 @@
129
130 /**
131  * @notice Claim redeemed assets
132  * @param redemptionId ID of the redemption
133  */
134 function claim(uint256 redemptionId) external {
135     address user = msg.sender;
136     Redemption storage redemption = redemptions[user][redemptionId];
137
138     if (redemption.user != user) revert NotYourRedemption();
139     if (redemption.processed) revert AlreadyProcessed();
140     if (block.timestamp < redemption.claimableAt) revert StillInQueue();
141
142     redemption.processed = true;
143     asset.safeTransfer(user, redemption.assets);
144
145     emit RedemptionClaimed(user, redemptionId, redemption.assets);
146 }
147
@@ 148,258 @@
259 }

```



Status

① Acknowledged



[WP-N11] The codebase contains logic to write to `Express.redemptionContract` , but lacks any corresponding read operations.

Issue Description

The `Express` redemption implementation does not utilize the `redemptionContract` variable.

It is unclear whether the code related to `redemptionContract` is legacy or dead code.

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L83>

```
83      address public redemptionContract;
```

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L225-L235>

```
225      /**
226       * @notice Set the redemption contract address
227       * @param _address Address of the redemption contract
228       */
229      function setRedemption(
230          address _address
231      ) external onlyRole(MAINTAINER_ROLE) {
232          if (_address == address(0)) revert InvalidAddress();
233          redemptionContract = _address;
234          emit SetRedemption(_address);
235      }
```

Status

 Acknowledged



[WP-N12] Unused errors

Issue Description

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/core/Token.sol#L22-L284>

```
22  contract Token is
23      Initializable,
24      ERC20Upgradeable,
25      ERC20PausableUpgradeable,
26      AccessControlEnumerableUpgradeable,
27      UUPSUpgradeable
28  {
    @@ 29,42 @@
43
44      error InvalidAddress();
45      error InvalidAmount();
46      error ExceedsIssueCap();
47      error CapBelowSupply();
48      error BannedSender(address sender);
49      error BannedRecipient(address recipient);
50      error InvalidBannedAccount(address account);
51      error BatchSizeTooLarge();
52
    @@ 53,283 @@
284 }
```

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L39-L826>

```
39  contract Express is
40      Initializable,
41      UUPSUpgradeable,
42      AccessControlEnumerableUpgradeable,
43      ExpressPausable,
44      MintRedeemLimiter
45  {
```

```

@@ 46,139 @@
140
141  /*/////////////////////////////////////////////////////////////////
142                               ERRORS
143  //////////////////////////////////////////////////////////////////////*/
144
145  error InvalidAddress();
146  error InvalidAmount();
147  error InvalidInput(uint256 input);
148  error UpdateTooEarly(uint256 timestamp);
149  error NotInKycList(address from, address to);
150  error BatchSizeTooLarge();
151  error InsufficientLiquidity(uint256 required, uint256 available);
152  error InsufficientOutput(uint256 received, uint256 minimum);
153  error MintLessThanMinimum(uint256 amount, uint256 minimum);
154  error FirstDepositLessThanRequired(uint256 amount, uint256 minimum);
155  error EmptyQueue();
156
@@ 157,825 @@
826  }

```

Status

✓ Fixed



[WP-I13] offRamp Can Withdraw Liquidity Committed to Pending Redemptions

Informational

Issue Description

`offRamp()` transfers underlying assets to treasury with no check against outstanding redemption queue liabilities. If operator calls `offRamp` after users have queued redemption requests, the contract may lack sufficient liquidity to process those redemptions.

Users' PRISM tokens are held in the Express contract pending processing. If `offRamp` drains the underlying balance, `processRedemptionQueue` will break at the liquidity check (line 481-485) and redemptions stall.

<https://github.com/OpenEdenHQ/openeden.prism.audit/blob/0e1710b02c2d813f0473e455774e18b61c64d354/contracts/extension/Express.sol#L697-L705>

```
697      /**
698       * @notice Transfer underlying from contract to treasury
699       * @param _amount Amount of underlying to transfer
700       */
701      function offRamp(uint256 _amount) external onlyRole(OPERATOR_ROLE) {
702          if (_amount == 0) revert InvalidAmount();
703          IERC20(underlying).safeTransfer(treasury, _amount);
704          emit OffRamp(treasury, _amount); // No check: _amount <= available -
              committed_to_queue
705      }
```

Redemption queue processing blocked due to insufficient liquidity. Users' PRISM is locked in the contract until operator replenishes funds. Requires OPERATOR_ROLE but no on-chain safeguard.

Recommendation

Track committed redemption liability and enforce: `require(balance - _amount >= totalQueueLiability)` .



Status

 Acknowledged



Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.